

# Polygons

A polyline is a chain of connected line segments. It is specified by giving the vertices (nodes)  $P_0, P_1, P_2 \dots$  and so on. The first vertex is called the initial or starting point and the last vertex is called the final or terminal point, as shown in the Fig. 3.1 (a). When starting point and terminal point of any polyline is same, i.e. when polyline is closed then it is called polygon. This is illustrated in Fig. 3.1 (b).

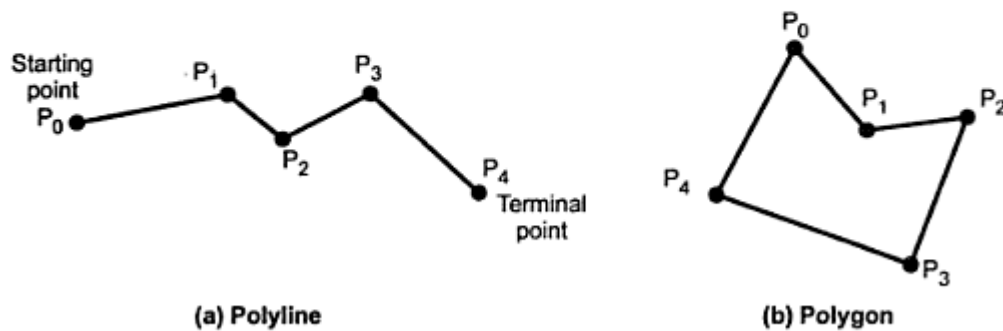


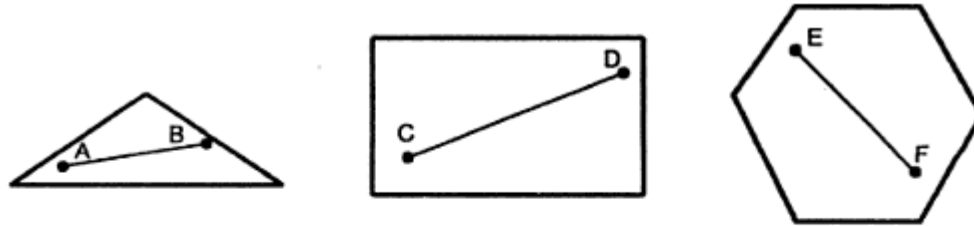
Fig. 3.1

## 3.2 Types of Polygons

The classification of polygons is based on where the line segment joining any two points within the polygon is going to lie. There are two types of polygons :

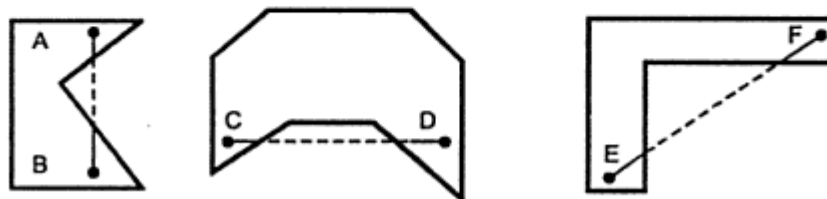
- Convex and
- Concave

A convex polygon is a polygon in which the line segment joining any two points within the polygon lies completely inside the polygon. The Fig. 3.2 shows the examples of convex polygons.



**Fig. 3.2 Convex polygons**

A concave polygon is a polygon in which the line segment joining any two points within the polygon may not lie completely inside the polygon. The Fig. 3.3 shows the examples of concave polygons.



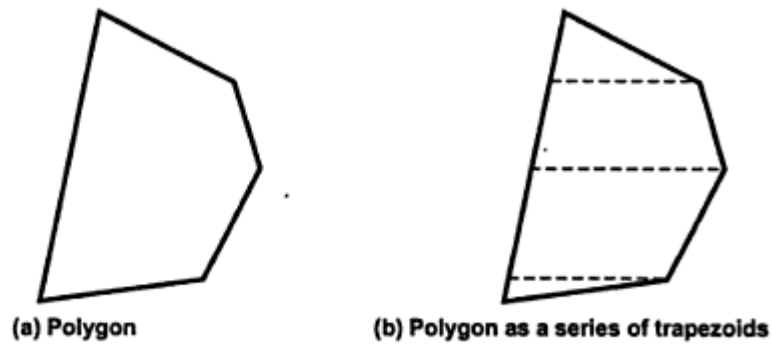
**Fig. 3.3 Concave polygons**

### 3.3 Representation of Polygons

We have seen that closed polyline is a polygon. Each polygon has sides and edges. The end points of the sides are called the polygon vertices. To add polygon to our **graphics** system, we must first decide how to represent it. There are three approaches to represent polygons according to the **graphics** systems :

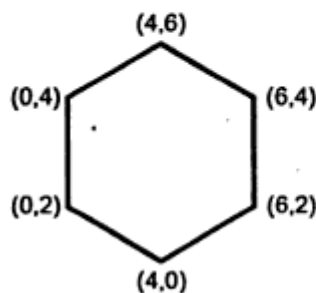
- Polygon drawing primitive approach
- Trapezoid primitive approach
- Line and point approach

Some **graphics** devices supports polygon drawing primitive approach. They can directly draw the polygon shapes. On such devices polygons are saved as a unit. Some **graphics** devices support trapezoid primitive. In such devices, trapezoids are formed from two scan lines and two line segments as shown in the Fig. 3.4. Here, trapezoids are drawn **by** stepping down the line segments with two vector generators and, for each scan line, filling in all the pixels between them. Therefore every polygon is broken up into trapezoids and it is represented as a series of trapezoids.



**Fig. 3.4 Representation of polygon**

Most of the other **graphics** devices do not provide any polygon support at all. In such cases polygons are represented using lines and points. A polygon is represented as a unit and it is stored in the display file. In a display file polygon can not be stored only with series of line commands because they do not specify how many of the following line commands are the part of the polygon. Therefore, new command is used in the display file to represent polygons. The opcode for new command itself specify the number of line segments in the polygon. The Fig. 3.5 shows the polygon and its representation using display file.



| DF_OP | DF_x | DF_y |
|-------|------|------|
| 6     | 0    | 2    |
| 2     | 0    | 4    |
| 2     | 4    | 6    |
| 2     | 6    | 4    |
| 2     | 6    | 2    |
| 2     | 4    | 0    |
| 2     | 0    | 2    |

**Fig. 3.5 Polygon and its representation using display file**

### 3.4 Entering Polygons

Let us see how to enter polygon command and data into the display file. We know that, we have to enter number of sides and the coordinates of the vertex points. This information can be entered using a following algorithm.

**Algorithm :** Entering the polygon into the display file

1. Read AX and AY of length N  
[AX and AY are arrays containing the vertices of the polygon and N is the number of polygon sides]
2.  $i = 0$  [Initialize counter to count number of sides]  
DF\_OP [i]  $\leftarrow$  N  
DF\_x [i]  $\leftarrow$  AX [i]  
DF\_y [i]  $\leftarrow$  AY [i]  
 $i \leftarrow i + 1$   
[Load polygon command]
3. do  
{  
DF\_OP [i]  $\leftarrow$  2  
DF\_x [i]  $\leftarrow$  AX [i]  
  
DF\_y [i]  $\leftarrow$  AY [i]  
 $i \leftarrow i + 1$   
}  
While ( $i < N$ ) [Enter line commands]
4. DF\_OP [i]  $\leftarrow$  2  
DF\_x [i]  $\leftarrow$  AX [0]  
DF\_y [i]  $\leftarrow$  AY [0]  
[Enter last line command]
5. Stop

### 3.5 An Inside Test

Once the polygon is entered in the display file, we can draw the outline of the polygon. To show polygon as a solid object we have to set the pixels inside the polygon as well as pixels on the boundary of it. Now the question is how to determine whether or not a point is inside of a polygon. One simple method of doing this is to construct a line segment between the point in question and a point known to be outside the polygon, as shown in the Fig. 3.6. Now count how many intersections of the line segment with the polygon boundary occur. If there are an odd number of intersections, then the point in question is inside; otherwise it is outside. This method is called the **even-odd method** of determining polygon inside points.

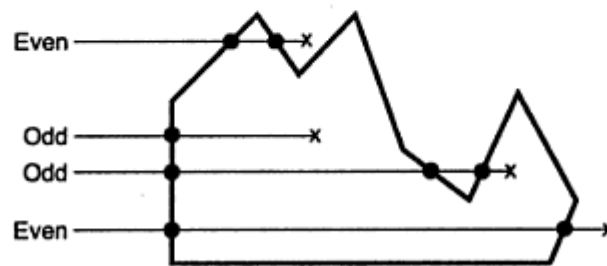


Fig. 3.6

If the intersection point is vertex of the polygon then we have to look at the other endpoints of the two segments which meet at this vertex. If these points lie on the same side of the constructed line, then the point in question counts as an even number of intersections. If they lie on opposite sides of the constructed line, then the point is counted as a single intersection. This is illustrated in Fig. 3.7.

Another approach to do the inside test is the **winding-number method**. Let us consider a point in question and a point on the polygon boundary. Conceptually, we can stretch a piece of elastic between these points and slide the end point on the polygon boundary for one complete rotation. We can then examine the point in question to see

how many times the elastic has wound around it. If it is wound at least once, then the point is inside. If there is no net winding, then the point is outside.

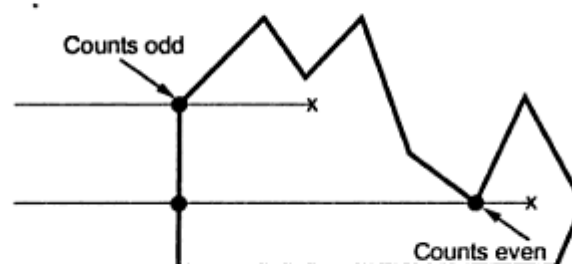


Fig. 3.7

Like even-odd method, in winding number method we have to picturise a line segment running from outside the polygon to the point in question and consider the polygon sides which it crosses. Here, instead of just counting the intersections, we have to give a **direction number** to each boundary line crossed, and we have to sum these direction numbers. The direction number indicates the direction the polygon edge was drawn relative to the line segment we have constructed for the test. This is illustrated in Fig. 3.8

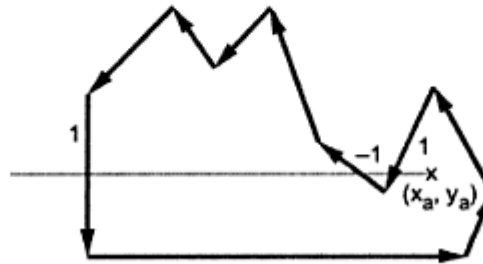


Fig. 3.8

As shown in the Fig. 3.8, point  $(x_a, y_a)$  is a test point and line  $y = y_a$  is a horizontal line runs from outside the polygon to point  $(x_a, y_a)$ . The polygon edges crossed by this line could be drawn in two ways. The edge could be drawn starting below the line, cross it, and end above the line or starting above the line, cross it, and end below the line. In first case we have to give direction number as  $-1$  and in the second case we have give direction number as  $1$ . After giving the direction numbers we have to take sum of these direction numbers which indicates whether the point in inside the polygon or not. The sum of the direction numbers for the sides that cross the constructed horizontal line segment is called the winding number for the point in question. For polygons or two dimensional objects, the point is said to be inside when the value of winding number is nonzero.

### 3.6 Polygon Filling

Filling the polygon means highlighting all the pixels which lie inside the polygon with any colour other than background colour. Polygons are easier to fill since they have linear boundaries.

There are two basic approaches used to fill the polygon. One way to fill a polygon is to start from a given "seed", point known to be inside the polygon and highlight outward from this point i.e. neighbouring pixels until we encounter the boundary pixels. This approach is called **seed fill** because colour flows from the seed pixel until reaching the polygon boundary, like water flooding on the surface of the container. Another approach to fill the polygon is to apply the inside test i.e. to check whether the pixel is inside the polygon or outside the polygon and then highlight pixels which lie inside the polygon. This approach is known as **scan-line algorithm**. It avoids the need for a seed pixel but it requires some computation. Let us see these two methods in detail.

### 3.6.1.2 Flood Fill Algorithm

Sometimes it is required to fill in an area that is not defined within a single colour boundary. In such cases we can fill areas **by** replacing a specified interior colour instead of searching for a boundary colour. This approach is called a flood-fill algorithm. Like boundary fill algorithm, here we start with some seed and examine the neighbouring pixels. However, here pixels are checked for a specified interior colour instead of boundary

colour and they are replaced **by** new colour. Using either a 4-connected or 8-connected approach, we can step through pixel positions until all interior point have been filled. The following procedure illustrates the recursive method for filling 8-connected region using flood-fill algorithm.



**Fig. 3.9**

**Procedure :** flood\_fill (x, y, old\_colour, new\_colour).

```
{
    if ( getpixel (x, y) = old_colour)
    {
        putpixel (x, y, new_colour);
        flood_fill (x + 1, y, old_colour, new_colour);
        flood_fill (x - 1, y, old_colour, new_colour);
        flood_fill (x, y + 1, old_colour, new_colour);
        flood_fill (x, y - 1, old_colour, new_colour);
        flood_fill (x + 1, y + 1, old_colour, new_colour);
        flood_fill (x - 1, y - 1, old_colour, new_colour);
        flood_fill (x + 1, y - 1, old_colour, new_colour);
        flood_fill (x - 1, y + 1, old_colour, new_colour);
    }
}
```

**Note :** 'getpixel' function gives the colour of specified pixel and 'putpixel' function draws the pixel with specified colour.

Comparison Chart



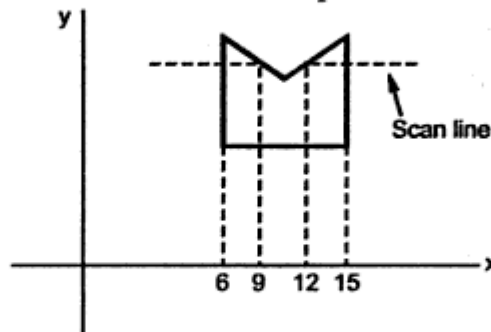
| BASIS FOR COMPARISON | FLOOD-FILL ALGORITHM                                                                                    | BOUNDARY-FILL ALGORITHM                                                         |
|----------------------|---------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| Basic                | It can have an area containing several colours.                                                         | It defines the area with a single colour.                                       |
| Painting process     | A random colour can be used to colour the interior portion then the old one is replaced with a new one. | Interior points are coloured by continuously searching for the boundary colour. |
| Memory consumption   | High                                                                                                    | Low                                                                             |
| Speed                | Comparatively slower                                                                                    | Fast                                                                            |
| Algorithm complexity | Simple relatively                                                                                       | Complicated                                                                     |

### 3.6.2 Scan Line Algorithm

Recursive algorithm for seed fill methods have got two difficulties : The first difficulty is that if some inside pixels are already displayed in fill colour then recursive branch terminates, leaving further internal pixels unfilled. To avoid this difficulty, we have to first change the colour of any internal pixels that are initially set to the fill colour before applying the seed fill procedures. Another difficulty with recursive seed fill methods is that it cannot be used for large polygons. This is because recursive seed fill procedures require stacking of neighbouring points and in case of large polygons stack space may be insufficient for stacking of neighbouring points. To avoid this problem more efficient method can be used. Such method fills horizontal pixel spans across scan lines, instead of proceeding to 4-connected or 8-connected neighbouring points. This is achieved by identifying the rightmost and leftmost pixels of the seed pixel and then drawing a horizontal line between these two boundary pixels. This procedure is repeated with changing the seed pixel above and below the line just drawn until complete polygon is filled. With this efficient method we have to stack only a beginning position for each horizontal pixel span, instead of stacking all unprocessed neighbouring positions around the current position.



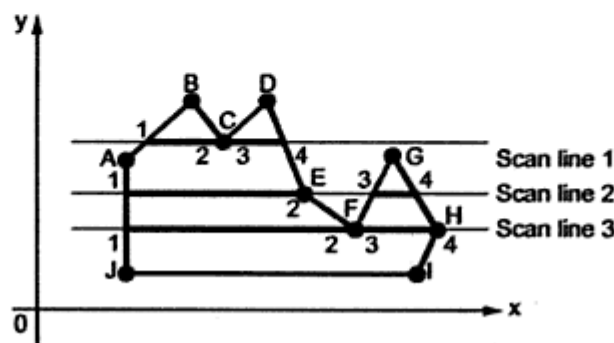
The Fig. 3.11 illustrates the scan line algorithm for filling of polygon. For each scan line crossing a polygon, this algorithm locates the intersection points of the scan line with the polygon edges. These intersection points are then sorted from left to right and the corresponding positions between each intersection pair are set to the specified fill colour.



**Fig. 3.11**

In Fig. 3.11, we can see that there are two stretches of interior pixels from  $x = 6$  to  $x = 9$  and  $x = 12$  to  $x = 15$ . The scan line algorithm first finds the largest and smallest  $y$  values of the polygon. It then starts with the largest  $y$  value and works its way down, scanning from left to right, in the manner of a raster display.

The important task in the scan line algorithm is to find the intersection points of the scan line with the polygon boundary. When intersection points are even, they are sorted from left to right, paired and pixels between paired points are set to the fill colour. But in some cases intersection point is a vertex. When scan line intersects polygon vertex a special handling is required to find the exact intersection points. To handle such cases, we must look at the other endpoints of the two line segments of the polygon which meet at this vertex. If these points lie on the same (up or down) side of the scan line, then the point in question counts as an even number of intersections. If they lie on opposite sides of the scan line, then the point is counted as single intersection. This is illustrated in Fig.3.12.



**Fig. 3.12 Intersection points along the scan line that intersect polygon vertices**

As shown in the Fig. 3.12, each scan line intersects the vertex or vertices of the polygon. For scan line 1, the other end points (B and D) of the two line segments of the polygon lie on the same side of the scan line, hence there are two intersections resulting two pairs : 1 - 2 and 3 - 4. Intersections points 2 and 3 are actually same points. For scan

For scan line 2 the other endpoints (D and F) of the two line segments of the polygon...

(D and F) of the two line segments of the polygon lie on the opposite sides of the scan line, hence there is a single intersection resulting two pairs : 1 - 2 and 3 - 4. For scan line 3, two vertices are the intersection points. For vertex F the other end points E and G of the two line segments of the polygon lie on the same side of the scan line whereas for vertex H, the other endpoints G and I of the two line segments of the polygon lie on the opposite side of the scan line. Therefore, at vertex F there are two intersections and at vertex H there is only one

We have seen that it is necessary to calculate x intersection points for scan line with every polygon side. We can simplify these calculations by using **coherence properties**. A coherence property of a scene is a property of a scene by which we can relate one part of a scene with the other parts of a scene. Here, we can use a slope of an edge as a coherence property. By using this property we can determine the x intersection value on the lower scan line if the x intersection value for current scan line is known. This is given as

$$x_{i+1} = x_i - \frac{1}{m}$$

where m is the slope of the edge

As we scan from top to bottom value of y coordinates between the two scan line changes by 1.

$$y_{i+1} = y_i - 1$$

Many times it is not necessary to compute the x intersections for scan line with every polygon side. We need to consider only the polygon sides with endpoints straddling the current scan line. See Fig. 3.13.

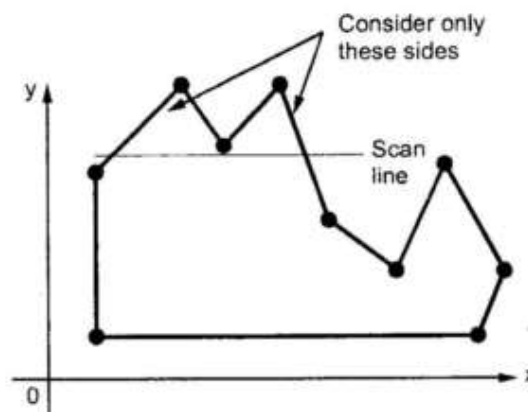
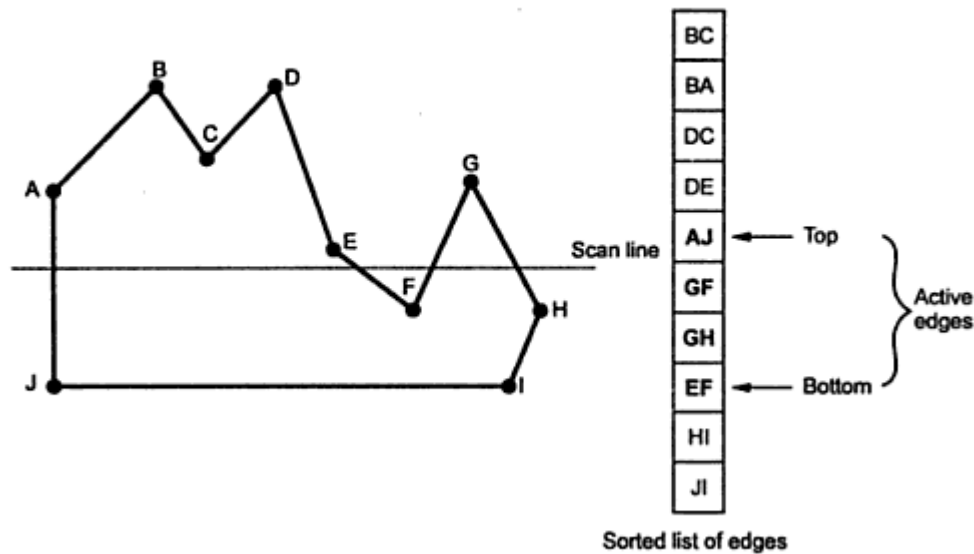


Fig. 3.13 Consider only the sides which intersect the scan line

It will be easier to identify which polygon sides should be tested for x-intersection, if we first sort the sides in order of their maximum y value. Once the sides are sorted we can process the scan lines from the top of the polygon to its bottom producing an active edge list for each scan line crossing the polygon boundaries. The active edge list for a scan line

line contains all edges crossed **by** that scan line. The Fig. 3.14 shows sorted edges of the polygon with active edges.



**Fig. 3.14 Sorted edges of the polygon with active edges**

In summary, a scan line algorithm for filling a polygon begins **by** ordering the polygon sides on the largest y value. It begins with the largest y value and scans down the polygon. For each y, it determines which sides can be intersected and finds the x values of these intersection points. It then sorts, pairs and passes these x values to a line drawing routine.

#### Scan Line Conversion Algorithm for Polygon Filling :

1. Read n, the number of vertices of polygon
2. Read x and y coordinates of all vertices in array x[n] and y[n].
3. Find  $y_{min}$  and  $y_{max}$ .
4. Store the initial x value ( $x_1$ ) y values  $y_1$  and  $y_2$  for two endpoints and x increment  $\Delta x$  from scan line to scan line for each edge in the array edges [n] [4].

While doing this check that  $y_1 > y_2$ , if not interchange  $y_1$  and  $y_2$  and corresponding  $x_1$  and  $x_2$  so that for each edge,  $y_1$  represents its maximum y coordinate and  $y_2$  represents its minimum y coordinate.

5. Sort the rows of array, edges [n] [4] in descending order of  $y_1$ , descending order of  $y_2$  and ascending order of  $x_2$ .
6. Set  $y = y_{max}$
7. Find the active edges and update active edge list :

if ( $y > y_2$  and  $y \leq y_1$ )  
     { edge is active }

else

{ edge is not active }

8. Compute the x intersects for all active edges for current y value [initially x-intersect is  $x_1$  and x intersects for successive y values can be given as

$$x_{i+1} \leftarrow x_i + \Delta x$$

$$\text{where } \Delta x = -\frac{1}{m} \text{ and } m = \frac{y_2 - y_1}{x_2 - x_1} \text{ i.e. slope of a line segment}$$

9. If x intersect is vertex i.e. x-intersect =  $x_1$  and  $y = y_1$  then apply vertex test to check whether to consider one intersect or two intersects. Store all x intersects in the x-intersect [ ] array.
10. Sort x-intersect [ ] array in the ascending order,
11. Extract pairs of intersects from the sorted x-intersect [ ] array.
12. Pass pairs of x values to line drawing routine to draw corresponding line segments
13. Set  $y = y - 1$
14. Repeat steps 7 through 13 until  $y \geq y_{\min}$ .
15. Stop

In step 7, we have checked for  $y \leq y_1$  and not simply  $y < y_1$ . Hence step 9 a becomes redundant. Following program takes care of that.